

UNIVERSIDAD TECNOLÓGICA NACIONAL - FACULTAD REGIONAL CÓRDOBA

INGENIERÍA DE SOFTWARE (ISW) — NOTAS DE CLASE TEÓRICAS

Materia: Ingeniería de Software | **Curso:** 4K1 | **Fecha:** 08 de Septiembre de 2026

Docentes: Ing. Mele (Titular) - Ing. Valeria (Adjunta/JTP)

¡ALERTA DE EXAMEN - ERROR RECURRENTE! Un producto **NO** es un conjunto de proyectos. Un producto está constituido por requerimientos y características. El ciclo de vida de un producto abarca en el tiempo múltiples proyectos sucesivos (por ejemplo, proyectos de versiones 1.0, 2.0, parches), pero el proyecto es la unidad de gestión y esfuerzo que crea o evoluciona dicho producto.

Diferenciación de Saberes y Roles:

- **Saberes de Producto:** Analistas funcionales, desarrolladores, arquitectos de software, líderes técnicos, testers, diseñadores UX/UI. Foco en construir la solución técnica (trabajado en materias como Diseño de Sistemas).
- **Saberes de Proceso:** Definición, estandarización, auditoría y mejora continua de procesos organizacionales.
- **Saberes de Proyecto:** Planificación, estimación, coordinación de equipos, negociación, monitoreo, control y gestión de riesgos. Este es el foco central de la materia ISW (roles de Líder de Proyecto / Project Manager).

4. Procesos Definidos vs. Procesos Empíricos

DESMITIFICACIÓN CRUCIAL: En los exámenes finales es común escuchar respuestas erróneas como: «Los procesos definidos son aquellos que ante la misma entrada obtienen la misma salida y no aceptan cambios». Esta afirmación es **totalmente falsa y absurda** en la vida profesional. Ningún cliente acepta que un equipo se niegue a cambiar un sistema porque «usa un proceso definido». La diferencia no es la negativa al cambio, sino la **filosofía y la actitud frente al cambio**.

Eje de Comparación	Procesos Definidos (Tradicional)	Procesos Empíricos (Ágil)
Premisa central	Asume que se puede prever y definir detalladamente de antemano qué hacer, quién lo hace, entradas, salidas y métricas.	Asume que el software es inherentemente complejo, ambiguo y cambiante; no se puede anticipar al 100%.
Extrapolabilidad	Considera que la experiencia es extrapolable entre proyectos mediante sistematización en métricas y datos históricos para lograr predictibilidad y repetibilidad.	Sostiene que la experiencia colectiva NO se extrapola: el desempeño depende del equipo específico, el cliente y el contexto puntual.
Actitud ante el cambio	Resistencia formal: los cambios exigen renegociación contractual, análisis de impacto en costo/tiempo y aprobación burocrática.	Abrazo al cambio: feedback rápido mediante iteraciones cortas (máximo 1 mes) de software funcional para minimizar el costo del error.
Mecanismo de gestión	Planificación exhaustiva anticipada, estandarización rigurosa, instanciación y adaptación formal justificada.	Inspección continua y adaptación basada en la experimentación y aprendizaje empírico del equipo.
Ciclo característico	Planificar -> Ejecutar -> Monitorear -> Comparar con línea base.	Asumir -> Construir -> Retroalimentar -> Revisar -> Adaptar.

Instanciación y Adaptación en Procesos Definidos: Un proceso organizacional definido es una pauta teórica general. Al inicio de cada proyecto, el líder debe **instanciar y adaptar** dicho proceso al contexto real (tamaño, complejidad, equipo). Omitir una actividad obligatoria (ej. documentar la arquitectura) requiere una justificación técnica formal rigurosa, nunca una omisión caprichosa.

Experiencia Individual vs. Colectiva: La experiencia individual de un profesional (diferencia entre junior y senior) siempre se conserva. Lo que el empirismo postula como no extrapolable es la **experiencia y velocidad colectiva del equipo**, dado que la dinámica social y contextual es irrepetible en otra organización.

5. Proceso vs. Ciclo de Vida del Proyecto

Existe una confusión conceptual arraigada entre ambos términos que la cátedra enfatizó aclarar:

- **PROCESO:** Define **QUÉ** actividades, métodos, prácticas, entradas, salidas y roles genéricos se requieren para construir software. Es la especificación teórica.
- **CICLO DE VIDA:** Determina **CÓMO Y CUÁNDO USAR** el proceso en el marco de un proyecto concreto. Define el orden secuencial o iterativo, cuánto de cada actividad se realiza en cada período y cuáles son los puntos de control o hitos.

Tipos Genéricos de Ciclo de Vida:

- **Secuencial (Cascada / Waterfall):** Ejecución lineal de etapas cerradas (100% de análisis antes de pasar al diseño, 100% de diseño antes de codificar).
- **Iterativo / Incremental:** Descompone el producto en porciones verticales ejecutables que atraviesan todas las disciplinas, generando incrementos sucesivos (ej. Ciclo del Proceso Unificado - RUP).
- **Recurso / Evolutivo:** Actividades que se anidan o despliegan dinámicamente según la emergencia del sistema.

Regla de Compatibilidad Fundamental:

- En **Procesos Definidos**, es válido utilizar **cualquiera de los tres tipos** de ciclo de vida (secuencial, iterativo o recursivo). *Nota histórica: * RUP fue concebido como iterativo incremental, pero muchas organizaciones fracasaron al forzarlo erróneamente sobre un ciclo en cascada.
- En **Procesos Empíricos**, **ÚNICAMENTE** es compatible el ciclo de vida **ITERATIVO**, debido a la necesidad impostergable de obtener feedback temprano y continuo.

6. Definición y Características de un Proyecto

Definición formal: Un proyecto es una **unidad de gestión del trabajo** orientada a alcanzar un objetivo concreto utilizando esfuerzo, tareas y recursos, con un resultado único y diferente cada vez.

Para que una iniciativa sea considerada proyecto, deben concurrir cuatro características indispensables:

- **1. Resultado Único y Distinto:** Cada producto de software es singular. No puede fabricarse en líneas de montaje continuo seriado (a diferencia de un control remoto o una pantalla física).
- **2. Duración Limitada (Temporal):** Posee fechas de inicio y fin claramente establecidas. Que se cumplan o desvíen depende de la gestión, pero la temporalidad es estricta.
- **3. Objetivo Claro y Alcanzable:**
 - **Claro:** Todo el equipo y los stakeholders comparten idéntica comprensión sin ambigüedades.
 - **Alcanzable:** Es factible de lograr con las capacidades, tiempo y recursos disponibles dando el mejor esfuerzo.
- **4. Tareas Interrelacionadas:** Mecanismo fundamental para dominar la complejidad: descomponer el trabajo global en tareas coordinadas con objetivos parciales que confluyen en el objetivo final.

Debate de Cátedra: ¿Objetivo Claro vs. Alcanzable?

- **¿Puede ser Claro y NO Alcanzable?** SÍ. Ocurre con frecuencia. Ejemplo: «Viajar en auto de Córdoba a Villa María en 15 minutos por Ruta 9». El objetivo es clarísimo, pero inviable físicamente. En software: «Desarrollar una billetera virtual completa con integraciones bancarias complejas en una semana».
- **¿Puede ser Alcanzable y NO Claro?** NO, NUNCA. Si el objetivo no está definido con claridad, es imposible determinar si es viable o no. Asumir un compromiso sin claridad equivale a firmar un cheque en blanco.

Tips Metodológicos de la Profesora Mele:

1. **Escucharse a uno mismo:** Pensar críticamente antes de hablar y corroborar si lo que se afirma es lógicamente razonable.
2. **Concepto riguroso de Ejemplo:** Un ejemplo no es una generalidad abstracta («una página web»), sino una *instancia concreta, contextualizada y específica* donde se demuestra con claridad el concepto teórico analizado (análogo a diseñar un caso de prueba).

7. La Planificación del Proyecto y su Valor

La profesora recordó la clásica máxima: «*Los planes no son nada; la planificación lo es todo*» (D. Eisenhower).

El verdadero valor de la gestión no reside en el documento estático (*Plan de Proyecto*), sino en el **proceso reflexivo de planificación**: el análisis de riesgos, la discusión de alternativas, el dimensionamiento de esfuerzo y los acuerdos asumidos por el equipo. Cuando se elabora un plan solo por formalismo burocrático o para completar un template, se transforma instantáneamente en **desperdicio**.

Construir software para uso masivo no equivale a «hacer un programita» casero: su escala, integración y criticidad exigen rigor metodológico de ingeniería.

8. Esencia y Accidente del Software — Frederick Brooks

A través del célebre ensayo «*No Silver Bullet: Essence and Accidents of Software Engineering*» y del clásico «*The Mythical Man-Month*», se analizaron los desafíos intrínsecos de la disciplina:

A. Dificultades Esenciales vs. Accidentales

- **Dificultades Accidentales:** Obstáculos vinculados a la realización técnica tangible en una época dada (limitaciones de memoria, sintaxis de lenguajes, lentitud de compiladores, herramientas primitivas). Los avances tecnológicos resuelven exitosamente estas trabas.
- **Dificultades Esenciales:** Problemas inherentes a la naturaleza conceptual del software. Brooks compara al software con un **Hombre Lobo**:
- **Complejidad:** No existen dos partes iguales; la escala incrementa exponencialmente los estados posibles.
- **Conformidad:** El software debe encajar forzosamente con instituciones, leyes, interfaces arbitrarias y sistemas preexistentes.
- **Mutabilidad:** Presión continua de cambio porque se percibe falsamente como infinitamente maleable.
- **Invisibilidad:** Carece de presencia física espacial o geométrica directa; no se puede «ver» en el espacio.

La Ilusión de la «Bala de Plata» y la Inteligencia Artificial:

Históricamente se creyó que cada innovación sería la salvación definitiva (así ocurrió con la Programación Orientada a Objetos en los 80/90). Hoy ocurre lo mismo con la **Inteligencia Artificial Generativa y los LLMs**. La IA genera avances notables sobre lo *accidental* (sintaxis rápida, autocompletado, borradores), pero **no resuelve la esencia** (comprender qué problema resolver, negociar requerimientos, asegurar consistencia conceptual). **Advertencia docente:** No incurrir en «delegación cognitiva». Usar la IA como herramienta, pero mantener juicio crítico propio; de lo contrario se entregan inconsistencias grotescas.

B. Ley de Brooks y Falacias de Gestión

- **Actividad humano-intensiva:** El costo predominante del software es el costo de las personas. En términos modernos debe medirse como **horas-persona lineales** (reemplazando el término arcaico «horas-hombre»).
- **Ley de Brooks:** «*Agregar personal a un proyecto retrasado solo consigue retrasarlo aún más*». Causas: los nuevos integrantes demandan capacitación; los desarrolladores sénior deben desviar su tiempo productivo para guiarlos; y los canales de comunicación crecen de forma combinatoria según la fórmula $N(N-1)/2$.
- **Restricciones secuenciales infranqueables:** «Nueve mujeres no pueden concebir un hijo en un mes». Ciertas actividades de ingeniería no son paralelizables.
- **El mito de las horas extras permanentes:** Forzar jornadas prolongadas genera fatiga mental en tareas de trabajo intelectual, lo que dispara exponencialmente la tasa de errores y defectos. El costo de retrabajo posterior supera con creces el retraso que se pretendía mitigar.

C. Principios de Diseño y Gestión de Brooks

- **Integridad Conceptual:** La calidad arquitectónica del producto debe estar guiada por una visión coherente centralizada (metáfora del equipo de cirujano de Brooks: un responsable clínico principal coordina las decisiones clave). «Organismo con dos cabezas se muere».
- **El Efecto del Segundo Sistema (Second-System Effect):** Tendencia peligrosa de los ingenieros a sobrecargar su segundo proyecto con complejidades innecesarias («ya que estamos, agreguemos tal cosa...»).
- **Principio YAGNI (You Aren't Gonna Need It) y Desperdicio:** Estadísticas mundiales revelan que el **45% de las funcionalidades desarrolladas en el software NUNCA son utilizadas** por los usuarios finales. Reducir este desperdicio es clave para optimizar recursos y plazos.

9. Administración de Proyectos y la Triple Restricción

La gestión o administración de proyectos abarca dos disciplinas inseparables:

- **Planificar:** Trazar la línea base, alcance, procesos, estimaciones y compromisos.
- **Monitorear y Controlar (Seguimiento):** Actividad de comparación continua. **No se puede controlar si no se planificó**, porque el

control exige un estándar o parámetro contra el cual comparar desviaciones.

La Triple Restricción (Triángulo de Hierro):

Cualquier proyecto se gobierna por tres dimensiones interdependientes: **Alcance** (requerimientos), **Tiempo** (cronograma) y **Costo / Recursos** (esfuerzo, presupuesto). En el centro neurálgico del triángulo se ubica la **CALIDAD**.

Alcance ↔ Tiempo ↔ Costo/Recursos ⇒ [CALIDAD en el centro]

Si una de las tres dimensiones se modifica sin ajustar equilibradamente las otras, la variable de sacrificio inmediata es la **Calidad** (se suprime análisis, se degrada la arquitectura y se recortan pruebas de software).

Dinámica de Negociación Profesional:

- El **Cliente** tiene la potestad de definir **qué necesita** (Alcance).
- El **Equipo de Proyecto / Líder** dimensiona técnicamente los **Recursos/Costos** y el **Tiempo**.
- Si el cliente pretende fijar rígidamente las tres variables (qué quiere, cuándo y cuánto pagar), el rol del líder queda anulado. Un líder profesional debe gestionar con margen sobre al menos dos variables. Si no hay más presupuesto ni tiempo, la **única negociación viable es reducir el Alcance** (eliminando el 45% prescindible mediante YAGNI).
- Se negocian **compromisos**, nunca estimaciones técnicas arbitrarias.

10. Estructura de Equipos: Tradicional vs. Ágil

Aspecto	Equipo Tradicional	Equipo Ágil
Liderazgo y autoridad	Líder de Proyecto / Project Manager que asigna tareas, planifica y toma decisiones unilateralmente.	Equipo autogestionado. No existe jefe que ordene tareas diarias; responsabilidad colectiva.
Estimación del trabajo	El Líder de Proyecto estima (a veces consulta a un experto externo que no programa).	Quienes realizan el trabajo son quienes estiman (principio no negociable).
Composición de perfiles	Silos funcionales especializados: analistas, programadores, testers, arquitectos aislados.	Equipos multidisciplinarios donde los miembros cubren el ciclo vertical completo de una User Story.
Interfaz con stakeholders	El Líder de Proyecto centraliza la comunicación con clientes, jefes y gerentes.	Colaboración directa entre el equipo técnico y el Product Owner (cliente integrado en el día a día).
Tamaño del equipo	Estructuras variables, frecuentemente extensas con jerarquías.	Equipos reducidos: 7 ± 2 personas (mínimo 5 para Scrum).

11. Componentes del Plan de Proyecto Tradicional

El Plan de Proyecto es la hoja de ruta que responde a cuatro interrogantes esenciales: ¿Qué? ¿Cuándo? ¿Cómo? ¿Quién?.

Componentes que deben especificarse en la planificación tradicional:

- **1. Objetivos del Proyecto:** Claros, medibles y alcanzables.
- **2. Alcance del Proyecto:** Todo el trabajo y solo el trabajo necesario para entregar el resultado.
- **3. Definición de Proceso y Ciclo de Vida:** Proceso adaptado y modelo de ciclo de vida seleccionado.
- **4. Estimaciones:** Tamaño, esfuerzo, calendario, recursos y costos.
- **5. Gestión de Riesgos:** Identificación, evaluación de exposición, mitigación y contingencia.
- **6. Asignación de Recursos:** Personas, roles, equipamiento e infraestructura.
- **7. Programación del Proyecto (Cronograma / Gantt):** Secuencia temporal de actividades.
- **8. Controles y Métricas:** Procedimientos de seguimiento y monitoreo de desvíos.

¡Aclaración! Elaborar un diagrama de Gantt **NO** es planificar el proyecto; es únicamente la última etapa donde se plasma cronológicamente la programación una vez resueltos el alcance, los procesos, las estimaciones y los recursos.

Alcance del Producto vs. Alcance del Proyecto

- **Alcance del Producto:** Conjunto de características, funciones y comportamientos que debe exhibir el software. Se especifica formalmente en la **Especificación de Requerimientos de Software (ERS)** o Documento de Requerimientos.
- **Alcance del Proyecto:** Todo el trabajo (y solo el trabajo) requerido para concebir, diseñar, programar, probar, desplegar y documentar dicho producto. Se especifica en el **Plan de Proyecto**.

Ejemplo de clase (Docente vs. Alumnos):

Si un cliente solicita: «Deseo un documento de requerimientos exhaustivo para evaluar la viabilidad de una app de gestión de agendas».

- **Alcance del Producto:** Permanece intacto (todas las funciones que la app tendría).
- **Alcance del Proyecto:** Cambia radicalmente: comprende únicamente el relevamiento, análisis y redacción del documento. No incluye codificación, arquitectura, testing ni despliegue.

12. El Proceso de Estimación en la Gestión Tradicional

A diferencia de las estimaciones ágiles (que son relativas y por comparación), la gestión tradicional busca **estimaciones cuantitativas absolutas** con alto foco en la precisión temprana.

El proceso de estimación tradicional sigue un **orden estricto y secuencial de derivación**:

1. TAMAÑO $\Rightarrow$ 2. ESFUERZO $\Rightarrow$ 3. CALENDARIO $\Rightarrow$ 4. COSTO

- **1. Tamaño del Software:** Mide la magnitud del producto a construir. Es independiente de quién lo desarrolle o del dinero disponible. Se cuantifica contando entidades tempranas (Casos de Uso por complejidad o requerimientos funcionales; las líneas de código [LOC] quedaron obsoletas para etapas tempranas).
- **2. Esfuerzo:** Cantidad de trabajo humano requerido expresado en **horas-persona lineales** (asumiendo a una sola persona ejecutando una tarea tras otra de forma continua). El resultado es una *bolsa de horas* (ej. 5.000 horas).
- **3. Calendario (Tiempo):** Derivación cronológica de la bolsa de horas en fecha concreta de entrega. Requiere calcular el paralelismo factible, la disponibilidad horaria del personal, las restricciones de secuencia y los días laborables.
- **4. Costo:** Es la última derivación, pues resulta de computar matemáticamente las variables previas:
 - **~80% del costo:** Esfuerzo humano (horas dedicadas multiplicadas por el costo/hora de cada perfil técnico).
 - **~10% del costo:** Conectividad e infraestructura técnica.
 - **~10% del costo:** Licencias de software, equipamiento, alquileres y gastos operativos.
 - **Costo emergente crítico:** Consumo de tokens de Inteligencia Artificial (llamadas a modelos fundacionales, que hoy representan facturas imprevistas considerables en las empresas).

Recursos Críticos: Aquellos elementos escasos o no reproducibles cuya disponibilidad condiciona directamente el cronograma y el presupuesto (ej. licencias concurrentes limitadas para herramientas CASE, un único sensor físico de hardware disponible para pruebas, cuotas acotadas de API de IA, ancho de banda restringido). No confundir con computadoras estándar de desarrollo.

13. Gestión Proactiva de Riesgos de Software

La gestión de riesgos es una disciplina obligatoria para evitar el fracaso del proyecto.

Distinción Crucial: Riesgo vs. Problema

- **Riesgo:** Evento potencial e incierto («un problema esperando suceder»). Posee una probabilidad matemática entre 0 y 1 ($0 < P < 1$).
- **Problema:** Hecho certero ($P = 1$). Si un equipo ya sabe que no llegará con el plazo establecido, **no es un riesgo, es un problema** que exige acción inmediata y renegociación.

Exposición del Riesgo (Risk Exposure)

Para homogeneizar y comparar riesgos de naturaleza heterogénea, se calcula la **Exposición**:

$$\text{Exposición} = \text{Probabilidad de Ocurrencia (P)} \times \text{Impacto del Daño (I)}$$

Este indicador cuantitativo permite priorizar qué riesgos atender primero: equilibra eventos de muy baja probabilidad pero daño catastrófico frente a eventos de altísima probabilidad y daño moderado.

Top Riesgos Habituales en Software

- **1. Requerimientos deficientes:** Ambigüedad, incompreensión de las necesidades del cliente y requerimientos cambiantes (el mayor causante histórico de fracasos).
- **2. Errores de estimación:** Subestimación sistemática de tamaño y esfuerzo.
- **3. Rotación de personal clave:** Renuncia de miembros críticos del equipo en momentos clave del ciclo de vida.
- **4. Riesgo tecnológico:** Adopción de tecnologías inmaduras, no probadas o desconocidas para el equipo de desarrollo.

Ciclo de Gestión Proactiva: Identificar tempranamente $\Rightarrow$ Analizar y cuantificar exposición $\Rightarrow$ Planificar acciones de mitigación y contingencia $\Rightarrow$ Monitorear y controlar $\Rightarrow$ Capitalizar aprendizaje institucional.

14. Monitoreo, Control y Métricas

El seguimiento y control exige una dinámica cíclica coherente:

ESTIMAR (Plan) $\Rightarrow$ MEDIR (Métricas) $\Rightarrow$ COMPARAR (Monitoreo) $\Rightarrow$ CORREGIR (Control)

- **Métricas de Proceso:** Datos sobre el desempeño y madurez de la organización.
- **Métricas de Proyecto:** Información para gobernar el proyecto en curso (esfuerzo real vs. estimado, avance de tareas, calendario).
- **Métricas de Producto:** Medición de la calidad técnica, defectos encontrados, complejidad ciclomática.

Principio de Simplicidad en Métricas:

«Si estamos a kilómetros de distancia del destino, carece de sentido medir en milímetros». No medir por el mero hecho de recolectar datos; cada métrica debe tener una justificación práctica y servir directamente para la toma de decisiones.